```
from flask import Flask, render template, request, redirect, session, url_for, flash
from werkzeug. utils import secure_filename
import os
import pandas as pd
import pickle
import lime
import lime. lime_text
from sklearn. feature_extraction.text import TfidfVectorizer
from sklearn. model_selection import train_test_split
from sklearn. naive_bayes import MultinomialNB
from sklearn. pipeline import make_pipeline
import sqlite3

app = Flask(__name__)
app. secret_key = 'your_secret_key'
UPLOAD_FOLDER = 'uploads'
os. makedirs (UPLOAD_FOLDER, exist_ok=True)
os. makedirs ('model', exist_ok=True)
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER

# ================== DB SETUP ====================
def init_sqlite_db ():
    conn = sqlite3.connect('users.db')
    conn. execute ('CREATE TABLE IF NOT EXISTS users (username TEXT, password
TEXT)')
    conn. close ()

init_sqlite_db ()

# ================== ROUTES ====================
@app. route ('/')
def home ():
    return redirect(url_for('login'))
```

```
@app. route ('/register', methods= ['GET', 'POST'])
def register ():
    if request. method == 'POST':
        username = request. form['username']
        password = request. form['password']
        with sqlite3.connect('users.db') as con:
            cur = con. cursor ()
            cur. execute ("INSERT INTO users (username, password) VALUES (?, ?)",
(username, password))
            con. commit ()
            flash ('Registration successful. Please login.')
            return redirect(url_for('login'))
    return render_template('register.html')

@app. route ('/login', methods= ['GET', 'POST'])
def login ():
    if request. method == 'POST':
        username = request. form['username']
        password = request. form['password']
        with sqlite3.connect('users.db') as con:
            cur = con. cursor ()
            cur. execute ("SELECT * FROM users WHERE username =? AND password =?",
(username, password))
            user = cur. fetchone ()
            if user:
                session['user'] = username
                return redirect(url_for('dashboard'))
            else:
                flash ('Invalid credentials')
    return render_template('login.html')

@app. route('/logout')
def logout ():
    session.pop ('user', None)
```

```
    flash ('Logged out successfully.')
    return redirect(url_for('login'))

@app. route('/dashboard')
def dashboard ():
    if 'user' not in session:
        return redirect(url_for('login'))
    return render_template('dashboard.html')

@app. route ('/upload', methods= ['GET', 'POST'])
def upload ():
    if 'user' not in session:
        return redirect(url_for('login'))
    if request. method == 'POST':
        file = request. files['dataset']
        if file and file. filename. endswith('.csv'):
            filepath = os. path. join(app.config['UPLOAD_FOLDER'], secure_filename (file.
filename))
            file. save(filepath)
            session['dataset'] = filepath
            flash ('Dataset uploaded successfully.')
            return redirect(url_for('train'))
        else:
            flash ('Please upload a valid CSV file.')
    return render_template('upload.html')

@app. route('/train')
def train ():
    if 'dataset' not in session:
        flash ('Please upload a dataset first.')
        return redirect(url_for('upload'))

    df = pd.read_csv(session['dataset'])
```

```
    # Clean up dataset
    df = df[['text', 'label']]. dropna ()
    df['text'] = df['text']. astype(str)
    df['label'] = df['label']. astype(str)

    print ("Label distribution:\n", df['label'].value_counts ())

    X = df['text']
    y = df['label']

    # Show unique labels
    print ("Unique labels:", y. unique ())

    X_train, X_test, y_train, y_test = train_test_split (X, y, test_size=0.2, random_state=42)

    model = make_pipeline (TfidfVectorizer (), MultinomialNB ())
    model.fit (X_train, y_train)

    # Evaluate model
    print ("Train accuracy:", model. score (X_train, y_train))
    print ("Test accuracy:", model. score (X_test, y_test))

    # Save model
    pickle. dump (model, open ('model/trained_model.pkl', 'wb'))

    flash ('Model trained and saved successfully.')
    return redirect(url_for('predict'))


@app. route ('/predict', methods= ['GET', 'POST'])
def predict ():
    if request. method == 'POST':
        text_input = request. form['text']
        model = pickle. load (open ('model/trained_model.pkl', 'rb'))
```

```
        prediction = model. predict([text_input]) [0]

        # Explainability with LIME
        explainer = lime. lime_text. LimeTextExplainer (class_names=model. classes_)
        exp = explainer.explain_instance(text_input, model. predict_proba, num_features=5)
        explanation_html = exp.as_html ()

        return render_template ('predict.html', prediction=prediction,
explanation=explanation_html, text=text_input)
    return render_template('predict.html')

# ================== MAIN ====================
if __name__ == '__main__':
    app.run(debug=True)
```